

Smart City Zero-Disk IoT Infrastructure



A complete backend data engineering platform with live ingestion, hot/cold storage, cloud runtime, and reviewer-ready reporting.

60

point final implementation package

18

slide enterprise demo story

The design blueprint is now an operational system: Go ingestion, TimescaleDB, Pub/Sub, GCS, BigQuery, GKE, CI/CD, Streamlit, Grafana, backups, restore tests, and cost controls.



Backend proof

live multi-source ingestion and durable storage paths



Reporting proof

Streamlit for city story, Grafana for operations



Ops proof

runtime health, release promotion, recovery, cost guardrails

Phase 3 proves the platform works across the full data lifecycle.

The project is positioned as an enterprise data platform: ingest, validate, store, operate, report, and recover.



Ingest

Go services poll simulator, air, weather, mobility, and water sources.



Store

TimescaleDB handles hot data; Parquet/GCS/BigQuery handle cold analytics.



Operate

GKE, CI/CD, promotion checks, backups, restore tests, and cost modes.



Report

Streamlit and Grafana create complementary reviewer surfaces.



Safety-first delivery

No secrets in package, guarded cloud mutation, protected public demo access.



Future-ready foundation

AI Agents can sit above the implemented telemetry and analytics layer.

The design blueprint became a runnable, observable implementation.



Phase 3 should not repeat Phase 2. It should show implementation depth, reviewer evidence, and operational maturity.

The system separates ingestion, storage, operations, and reporting concerns.

This keeps the demo understandable while preserving real backend/data-engineering depth.



Multiple real city domains feed one normalized reading model.

The platform supports deterministic local review and live-source proof when network/API access is available.

Air quality - OpenAQ



API-key gated

Weather - Open-Meteo



network live

Mobility - Divvy GBFS



station live

Water - USGS



river gage

Simulator



deterministic local

Reviewer promise:
No fake Grafana rows.
No synthetic cloud
metrics.
Live sources degrade
gracefully.

Validation and quality flags make the city telemetry reviewable.

Phase 3 evidence is stronger because the platform does not treat every incoming payload as trustworthy by default.



Normalization

Source-specific payloads become a common reading shape with metric, value, timestamp, and location.



Validation

Invalid, suspect, and valid quality flags protect analytics and dashboard interpretation.



Observability

Ingestion metrics surface throughput, freshness, channel fill, and dropped readings.



Coverage

Source and metric coverage tables show what data is actually present.



Traceability

Tables retain sensor IDs, sources, metric names, and reading freshness.

TimescaleDB is the operational store for current city state.

It holds readings and ingestion metrics that power both local dashboards and runtime validation.

Hot data responsibilities

- sensor_readings for city telemetry
- ingestion_metrics for platform operations
- row checks for source coverage
- tables powering Streamlit and Grafana

Reviewer commands

make run-local
make seed-simulator
make poll-multisource-once
make grafana-demo-ready

Parquet, GCS, and BigQuery turn hot readings into historical evidence.

The cold path supports longer-term reporting while keeping the live dashboard focused on current operations.



Local Parquet

make export-cold-demo writes partitioned files from TimescaleDB rows.



GCS upload

make export-cold-gcs promotes cold files into cloud object storage.



BigQuery view

make bigquery-cold-check validates external-table visibility.



Streamlit evidence

The command center can show local/cloud cold-path proof.



Retention-safe

Exports do not delete hot TimescaleDB rows during demo validation.

The runtime path proves the platform can operate beyond a laptop.

GKE Autopilot, Workload Identity, internal TimescaleDB, CronJobs, and private backend services form the cloud operating model.



CI/CD and promotion checks create a controlled release story.

The implementation has local builds, cloud image publishing, and manual runtime promotion rather than ad hoc deployment.



Local image proof

make docker-build and make docker-smoke validate deployable service images.



GitHub Actions

OIDC-backed workflows publish latest-main and short-SHA tags.



Runtime promotion

Manual promotion deploys selected tags and verifies release health.



Workflow checks

make ci-cd-check guards publish and promotion workflow integrity.



Image checks

make runtime-image-check verifies selected tags exist before rollout.

The runtime has health, backup, and restore proof.

These checks make the project feel operated, not merely launched.



Runtime health

Pods, jobs, PVCs, logs, images, and latest object checks.



Backups

GKE job exports TimescaleDB state to GCS for recovery evidence.



Restore test

Disposable namespace validates backup integrity without disturbing production.



Evidence capture

make runtime-evidence writes sanitized reviewer output.



Demo recovery

runbooks show how to start, stop, idle, resume, and clean up safely.

Demo operations are explicit, reversible, and cost-aware.

The runtime can be placed in demo, idle, and resume modes without deleting persistent storage.

Demo mode

resume workloads and CronJobs for a review window

Idle mode

disable public ingress and scale optional workloads down

Resume mode

bring the runtime back and validate health

Cost report

summarize active resources and public demo posture

Streamlit is the polished public command center.

It turns backend data into a product-like review experience for city operations and historical evidence.

Reviewer-facing views

- Executive KPIs
- Air, weather, water, mobility reporting
- Source health and coverage
- Sensor map and latest readings
- Historical archive evidence

Demo commands

`make run-streamlit`

or guarded public demo:
`ALLOW_PUBLIC_INGRESS=yes`
`make public-demo-apply`
`make public-demo-url`

Grafana is the enterprise operations room for the data platform.

It complements Streamlit by emphasizing ingestion health, sensor estate, geomap, coverage, and data quality.



Executive overview

readings, sensors, source coverage, valid rate, freshness, drops.



Ingestion operations

queue throughput, backpressure channel fill, metric freshness.



Sensor estate

metric coverage, latest readings, freshness table, native geomap.



City domains

air quality, weather, water, and mobility trend panels.

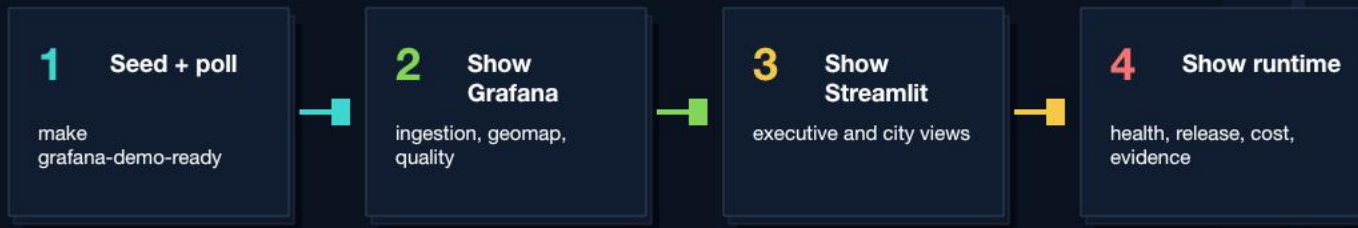


Quality lens

quality distribution, valid rate by source, suspect/invalid review.

The live review path moves from backend proof to reporting proof.

This sequence keeps the presentation practical and avoids re-litigating Phase 2 design details.



Reviewer access is useful without exposing the platform unsafely.

The implementation includes practical guardrails around secrets, public access, cloud mutation, and recovery.



No secret package

.env, Terraform state, local data, caches, and artifacts are excluded.



Protected demo

Public ingress exposes only Streamlit and requires a demo password.



Guarded changes

Terraform apply and public ingress require explicit allow flags.



Private backend

Backend services remain inside the runtime network.



Recoverability

Backup and restore checks prove data recovery posture.

AI Agents are the next layer above the implemented telemetry platform.

Frame this as future scope unless teammate code lands before final packaging, then refresh the deck and zip.



Incident triage

Summarize abnormal readings and recommend operator actions.



Anomaly explanation

Connect air, mobility, weather, and water signals into a cause hypothesis.



Operator copilot

Answer questions over TimescaleDB and BigQuery history.



Policy-aware routing

Route alerts with safety, privacy, and access boundaries.

The final package is reproducible, reviewable, and ready to refresh.

If AI Agent work lands later, pull latest main, update final materials, rerun checks, and rebuild the zip.

Final commands

```
make phase3-check  
make phase3-package  
make phase3-package-list
```

Submit

dist/Project_Phase_3_Group4.zip

Deck backup:
docs/final/Project_Phase_3_Group4_Presentation.pdf